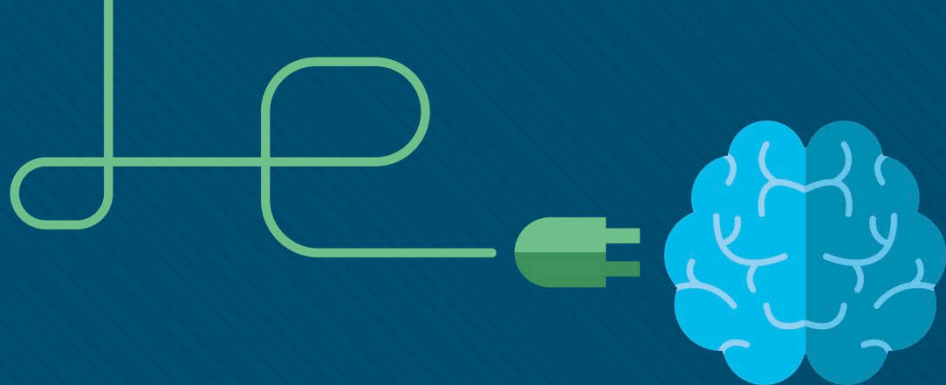




Week 12

RH134 Chapter 7 Manage Basic Storage
RH134 Chapter 8 Manage Storage Stack



Manage Basic Storage

1 Introduction

- Disk partitions can be created either during the installation process or at any time after the installation is complete. The competent Linux system administrator should be skilled sufficiently to create partition at both situations.
- The partition creation process during installation is typically GUI-based while the technique to create partitions after the installation stage is typically a CLI-based program. The configuration of partition mounting is fairly automated when partitions are created during installation while the latter is manually configured.
- Although most Linux administrators prefer to create partitions during the installation process, it is still important to be equipped with the know-how to create partitions after installation for the following reasons:
 - There could be cases where a second hard drive is added after the system has been running for a while
 - Repartitioning may be necessary in response to an audit or internal review exercise where optimal partitioning to improve system availability is proposed.

2 Partitioning during Installation

- The screen capture below shows part of the installation summary screen when installing a new Red Hat 9 server. It shows that by default automatic partitioning is selected. With automatic partitioning, only / and /boot partitions are created.
- To manually configure partitions, click the Installation Destination tab and you will be presented with the Installation Destination dialog. Select “I will configure partitioning” in the “Other Storage Options” section.



- Click “Done” and you will be brought to the “Manual Configuration” screen.

Other Storage Options

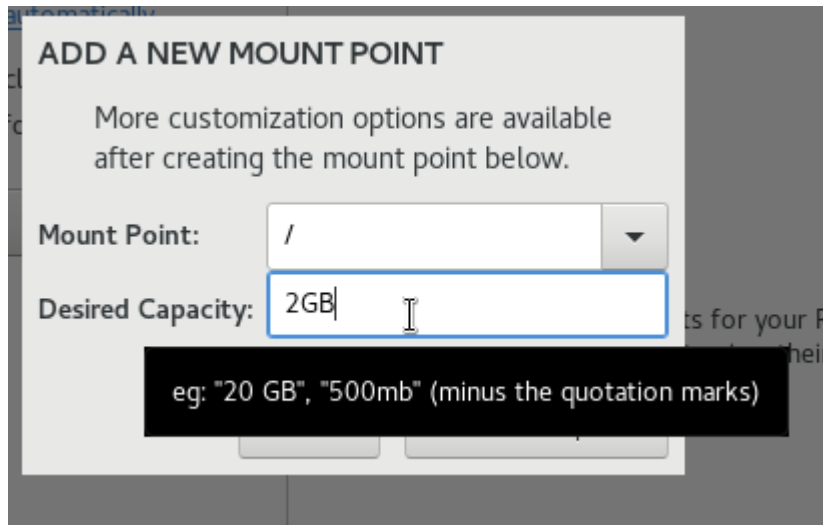
Partitioning

☐ Automatically configure partitioning. ☒ I will configure partitioning.

☐ I would like to make additional space available. -----

2 Partitioning during Installation (Cont'd)

- Add a new partition by clicking the “+” button as shown on the right.
- As depicted below, you can use the drop down list and select a default mount point (part of FHS) and specify a desired size for the mount point.



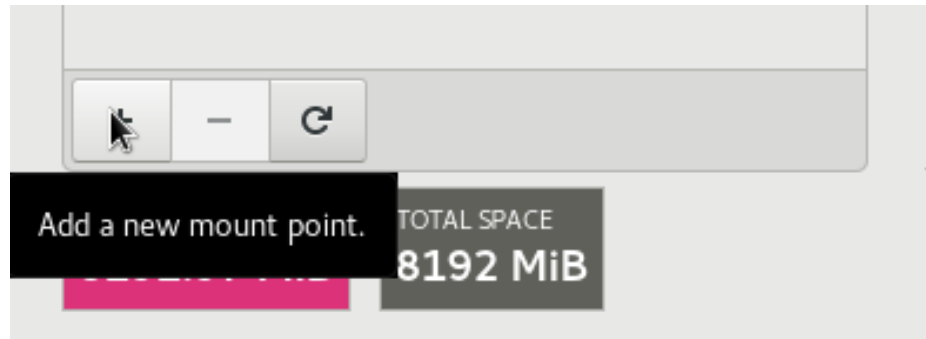
ADD A NEW MOUNT POINT

More customization options are available after creating the mount point below.

Mount Point:

Desired Capacity:

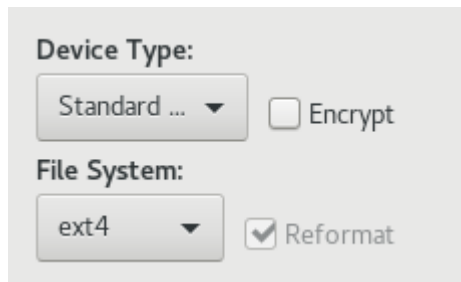
eg: "20 GB", "500mb" (minus the quotation marks)



- The example on the left shows that the / mount point has been selected and the desired capacity is 2 gigabytes.

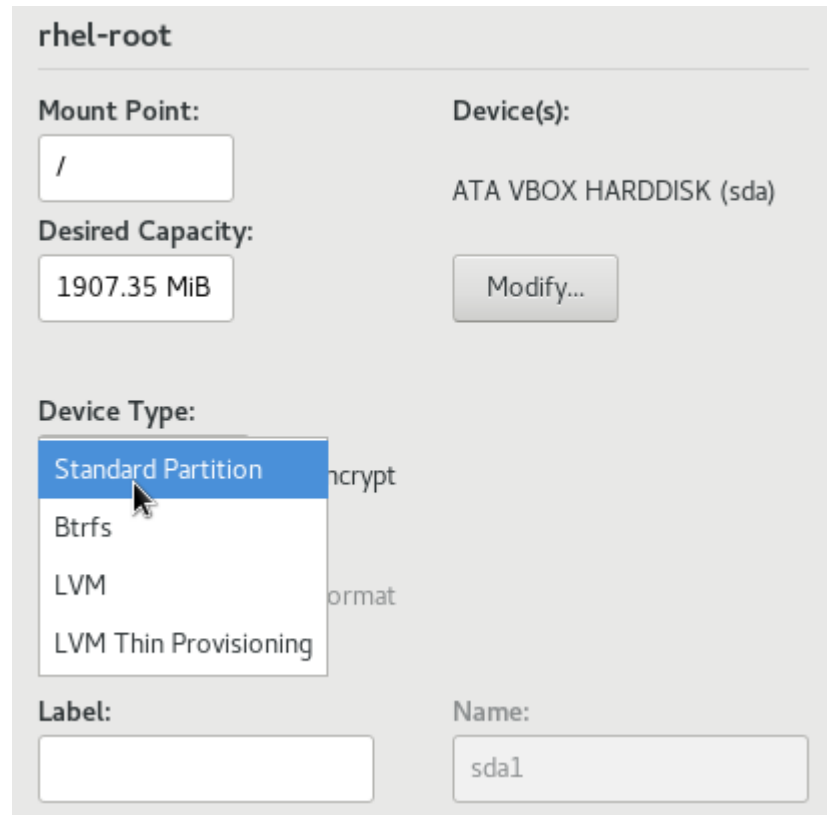
2 Partitioning during Installation (Cont'd)

- You can then select the “Device Type” and the filesystem format. In the example below, “Standard Partition” has been selected as the “Device Type” and the ext4 selected as filesystem format.
- You also have the option to Encrypt the disk for increased security but that may adversely impact performance and the secrecy of the data should be weighed against the performance requirements.
- Using the same steps, you can create the rest of the partitions as required. Click “Done” to exit “Manual configuration” dialog.



Device Type:
Standard ... ☐ Encrypt

File System:
ext4 ☒ Reformat



rhel-root

Mount Point: /

Device(s): ATA VBOX HARDDISK (sda)

Desired Capacity: 1907.35 MiB Modify...

Device Type:
Standard Partition
Btrfs
LVM
LVM Thin Provisioning

Label:

Name: sda1

3 Creating Partitions After Installation

- The most common command line tool for editing the partition tables on disks is called fdisk. This command can be used to create, modify, delete and list the partitions on a hard drive.
- All changes made during the use of the fdisk tool is not effected until they have been written (committed). If you made errors during the use of the tool, simply quit and restart.
- The fdisk programme has an interactive and non-interactive mode. The interactive mode is used to create, modify and delete partitions while the non-interactive mode is used to list the partitions. The fdisk program requires root privileges to run.
- By default, Linux technically partitions by sector, so using the -u option is not really required. To display output in cylinders, use the -u=cylinders option.

3.1 Displaying Partitions

- To use fdisk in its non-interactive mode, add the -l option as shown below:

```
sysadmin@localhost:~$ fdisk -cul
Disk /dev/sda: 21.5 GB, 21474836480 bytes
255 heads, 63 sectors/track, 2610 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x000571a2

   Device Boot      Start         End      Blocks   Id  System
/dev/sda1  *           1         2481     19921920   83   Linux
/dev/sda2             2481         2611       1046529    5   Extended
/dev/sda5             2481         2611       1046528   82   Linux swap / Solaris
```

- Important sections of the output that you need to understand are:

1. Device Info

```
Disk /dev/sda: 21.5 GB, 21474836480 bytes
```

3.1 Displaying Partitions (Cont'd)

2. Columns

Device	Boot	Start	End	Blocks	Id	System
/dev/sda1	*	1	2481	19921920	83	Linux
/dev/sda2		2481	2611	1046529	5	Extended
/dev/sda5		2481	2611	1046528	82	Linux swap / Solaris

Device	The specific partition that the row is describing. For example, /dev/sda1 is the first partition on the first SATA hard drive.
--------	--

Start	The starting sector of the partition.
-------	---------------------------------------

End	The ending sector of the partition.
-----	-------------------------------------

Blocks	The size of the partition in blocks.
--------	--------------------------------------

Id	An identifier which is used to tell the kernel what type of filesystem should be placed on this partition. For example, the value 83 indicates that this partition should have an ext2, ext3, or etx4 filesystem type.
----	--

System	A human-readable name that indicates the type of filesystem the Id column refers to. For example, 83 is a Linux filesystem.
--------	---

3.2 fdisk Interactive Mode

- In the interactive mode, a system administrator can use the fdisk command to create and modify partitions. To enter interactive mode, do not use the -l option, but still use the -c and -u options. The pathname for the disk to edit is required. For example, to edit the partitions for the first SATA /dev/sda hard drive, execute the following command on the right to display the prompt and type m to display the option menu.

```
[root@localhost ~]# fdisk -u=sectors -c /dev/sda
Welcome to fdisk (util-linux 2.23.2).

Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Command (m for help): m
Command action
 a   toggle a bootable flag
 b   edit bsd disklabel
 c   toggle the dos compatibility flag
 d   delete a partition
 g   create a new empty GPT partition table
 G   create an IRIX (SGI) partition table
 l   list known partition types
 m   print this menu
 n   add a new partition
 o   create a new empty DOS partition table
 p   print the partition table
 q   quit without saving changes
 s   create a new empty Sun disklabel
 t   change a partition's system id
 u   change display/entry units
 v   verify the partition table
 w   write table to disk and exit
 x   extra functionality (experts only)

Command (m for help):
```

3.2 fdisk Interactive Mode (Cont'd)

- Before making changes using the fdisk interactive mode, it is a good practice to print out the partition table using the p option.

```
Command (m for help): p

Disk /dev/sda: 8589 MB, 8589934592 bytes, 16777216 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk label type: dos
Disk identifier: 0x000f3785

   Device Boot      Start         End      Blocks    Id  System
/dev/sda1  *        2048       1026047       512000    83  Linux
/dev/sda2             1026048       5220351      2097152    83  Linux
/dev/sda3             5220352       9414655      2097152    82  Linux swap / Solaris
/dev/sda4             9414656      16777215       3681280     5  Extended
/dev/sda5             9416704      11513855       1048576    83  Linux
/dev/sda6            11515904      16775167       2629632    83  Linux
```

- Notice from the output that there are some space left in the hard drive. The ending sector is 16775167 while /dev/sda has a total of 16777216 sectors. The difference is only about 1MB which is minimal.

3.3 Creating Partitions

- In order to create a new partition, the n command action should be chosen:

```
Command (m for help): n
Partition type:
   p   primary (0 primary, 0 extended, 4 free)
   e   extended
Select (default p):
```

- Recall that with MBR, there is a limit of 4 primary partitions or 3 primary partitions and 1 extended partitions, within which multiple logical partitions can be created.
 - If no extended partition has been created, the choices will be e for an extended partition or p for a primary partition
 - If the extended partition has been created, the fdisk utility will automatically create a logical partition within the free space of the extended partition.
- When creating an extended partition, allocate all of the remaining free sectors to the extended partition and then logical partitions can be created within it.

3.3 Creating Partitions (Cont'd)

- In the example here, a primary partition of size 2 gigabytes has been created and given a partition number of 1. We have taken the default first sector of 2048.

```
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-33554431, default 2048):
Using default value 2048
Last sector, +sectors or +size{K,M,G} (2048-33554431, default 33554431): +2G
Partition 1 of type Linux and of size 2 GiB is set
```

```
Command (m for help): p
```

```
Disk /dev/sdb: 17.2 GB, 17179869184 bytes, 33554432 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk label type: dos
Disk identifier: 0xdcda9c51
```

Device	Boot	Start	End	Blocks	Id	System
/dev/sdb1		2048	4196351	2097152	83	Linux

3.3 Creating Partitions (Cont'd)

- We continue the example after creating three primary partitions by creating the fourth partition as an extended partition. Notice we have allocate the remaining unallocated drive space to the

```
Command (m for help): n
Partition type:
  p   primary (3 primary, 0 extended, 1 free)
  e   extended
Select (default e): e
Selected partition 4
First sector (12584960-33554431, default 12584960):
Using default value 12584960
Last sector, +sectors or +size{K,M,G} (12584960-33554431, default 33554431):
Using default value 33554431
Partition 4 of type Extended and of size 10 GiB is set
```

```
Command (m for help): p
```

```
Disk /dev/sdb: 17.2 GB, 17179869184 bytes, 33554432 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk label type: dos
Disk identifier: 0xdcda9c51
```

Device	Boot	Start	End	Blocks	Id	System
/dev/sdb1		2048	4196351	2097152	83	Linux
/dev/sdb2		4196352	8390655	2097152	83	Linux
/dev/sdb3		8390656	12584959	2097152	83	Linux
/dev/sdb4		12584960	33554431	10484736	5	Extended

extended partition by simply pressing Enter when prompted for the size of the extended partition.

3.4 Changing the Filesystem Type

- To change the file system type, use the `t` command option:

```
Command (m for help): t
Partition number (1-8, default 8): 3
Hex code (type L to list all codes):
```

- To see a list of the different file system type that you can set to, type `L`
- To set partition 3 to a Linux swap file system type, enter 82.

```
Hex code (type L to list all codes): 82
Changed type of partition 'Linux' to 'Linux swap / Solaris'

Command (m for help): p
```

Device	Boot	Start	End	Blocks	Id	System
/dev/sdb1		2048	1026047	512000	83	Linux
/dev/sdb2		1026048	5220351	2097152	83	Linux
/dev/sdb3		5220352	9414655	2097152	82	Linux swap / Solaris
/dev/sdb4		9414656	33554431	12069888	5	Extended

Hex code (type L to list all codes): L

0	Empty	24	NEC DOS	81	Minix / old Lin	bf	Solaris
1	FAT12	27	Hidden NTFS Win	82	Linux swap / So	c1	DRDOS/sec (FAT-
2	XENIX root	39	Plan 9	83	Linux	c4	DRDOS/sec (FAT-
3	XENIX usr	3c	PartitionMagic	84	OS/2 hidden C:	c6	DRDOS/sec (FAT-
4	FAT16 <32M	40	Unix 80286	85	Linux extended	c7	Syrinx
5	Extended	41	PPC PREP Boot	86	NTFS volume set	da	Non-FS data
6	FAT16	42	SFS	87	NTFS volume set	db	CP/M / CTOS / .
7	HPFS/NTFS/exFAT	4d	QNX4.x	88	Linux plaintext	de	Dell Utility
8	AIX	4e	QNX4.x 2nd part	8e	Linux LVM	df	BootIt
9	AIX bootable	4f	QNX4.x 3rd part	93	Amoeba	e1	DOS access
a	OS/2 Boot Manag	50	OnTrack DM	94	Amoeba BBT	e3	DOS R/O
b	W95 FAT32	51	OnTrack DM6 Aux	9f	BSD/OS	e4	SpeedStor
c	W95 FAT32 (LBA)	52	CP/M	a0	IBM Thinkpad hi	eb	BeOS fs
e	W95 FAT16 (LBA)	53	OnTrack DM6 Aux	a5	FreeBSD	ee	GPT
f	W95 Ext'd (LBA)	54	OnTrackDM6	a6	OpenBSD	ef	EFI (FAT-12/16/
10	OPUS	55	EZ-Drive	a7	NeXTSTEP	f0	Linux/PA-RISC b
11	Hidden FAT12	56	Golden Bow	a8	Darwin UFS	f1	SpeedStor
12	Compaq diagnost	5c	Priam Edisk	a9	NetBSD	f4	SpeedStor
14	Hidden FAT16 <3	61	SpeedStor	ab	Darwin boot	f2	DOS secondary
	FAT16	63	GNU HURD or Sys	af	HFS / HFS+	fb	VMware VMFS
	HPFS/NTF	64	Novell Netware	b7	BSDI fs	fc	VMware VMKCORE
	rtSleep	65	Novell Netware	b8	BSDI swap	fd	Linux raid auto
	W95 FAT3	70	DiskSecure Mult	bb	Boot Wizard hid	fe	LANstep
				be	Solaris boot	ff	BBT

3.5 Saving Changes

- To save (commit) the settings which you have made using fdisk, use the w command option.

```
Command (m for help): w
The partition table has been altered!

Calling ioctl() to re-read partition table.
Syncing disks.
```

- This error is the result of the kernel being unable to read the new partition table into memory from the MBR, something that the ioctl() function is responsible for. As mentioned in the output of this error, the partprobe or kpartx command can be executed to fix this issue if these commands are installed on the system. If not, then the system will need to be rebooted before the new partitions can be used.

4 Managing GPT

- Some hard drives make use of a partitioning technology called **Master Boot Record (MBR)** while others make use of a partitioning type called **GUID Partitioning Table (GPT)**. The MBR type of partitioning has been used since the early days of the personal computer (PC), and the GPT type has been available since the year 2000.
- The GPT disks use a newer type of partitioning, which allows the user to divide the disk into more partitions than what MBR supports. GPT also allows having partitions, which can be larger than two terabytes (MBR does not). The tools for managing GPT disks are named similarly to the **fdisk** counterparts: the **gdisk**, **cgdisk**, and **sgdisk** programs.
- To create and manage GPT partitions from the command line, you can use the **gdisk** utility, also called GPT **fdisk**. It operates in a similar fashion to **fdisk** except it operates on GPT partitions and requires the device to be specified in order to work.

4 Managing GPT (Cont'd)

- An example of the use of gdisk.

```
sysadmin@localhost:~$ sudo gdisk /dev/sdb1
GPT fdisk (gdisk) version 1.0.3

Partition table scan:
MBR: not present
BSD: not present
APM: not present
GPT: not present

Creating new GPT entries.

Command (? for help): ?
b          back up GPT data to a file
c          change a partition's name
d          delete a partition
i          show detailed information on a partition
l          list known partition types
n          add a new partition
o          create a new empty GUID partition table (GPT)
p          print the partition table
q          quit without saving changes
```

5 Logical Volume Management

- **Logical Volume Management (LVM)** is a method of managing hard disk space that provides more flexibility than traditional partitioning of disks. LVM was developed specifically to address some of the limitations of traditional partitioning. To understand this, consider the following scenarios:

Scenario #1

Synopsis: A system administrator performs a fresh installation of what will become a production machine. The administrator chooses to implement the following partitioning scheme: / is 2000MB, /var is 500MB, and /home is 1000MB.

The intent behind this partitioning scheme was to provide plenty of space for the four users who will be using the system. By making /home a separate partition, the administrator is also able to implement disk quotas.

The Problem: For the first few months, the system performs perfectly. However, errors start to occur. Users are unable to print. After some research, the administrator discovers that the /var partition is now full of log file entries. Unfortunately, this is also where print jobs are sent prior to being sent to the printer.

5 Logical Volume Management (Cont'd)

Scenario #1 (Cont'd)

The obvious solution is to make the /var partition bigger. With traditional partitions, this can be a nightmare that may require downtime for the system and perhaps even a reinstallation. One of the great benefits of LVM is the ability to increase the size of the partition, referred to as a *volume* in LVM, as needed without the necessity for system downtime.

Scenario #2

Synopsis: A system administrator adds two new hard drives to the system. One hard drive is 40GB and the second is 20GB. The requirements for this system are to place a very large database on a single partition that is at least 60GB in size. This isn't possible with traditional partitions in this scenario because a partition must reside on a single hard drive.

With LVM, a volume (think partition) can span over multiple hard drives. This provides a great deal more flexibility over traditional partitions and is necessary for today's big data environments.

5 Logical Volume Management (Cont'd)

Scenario #3

Synopsis: A system administrator wants to back up the data in the /home partition. In order to get a clean backup, all users must be signed off the system, and the /home partition needs to be unmounted, resulting in downtime. This is due to the way that filesystem backup programs work: they back up the filesystem metadata first, then back up the file contents. With a live filesystem, changes to files during the backup results in possibly corrupted backups that may be worthless.

The solution is to make use of a feature called an LVM snapshot. An administrator can use this snapshot to back up a filesystem that will appear to be static, but won't require any system downtime.

5.1 LVM Concepts

The steps to implement LVM include the following:

1. Connect the physical devices to the system.
2. Use *pvcreate* to convert the desired devices into physical volumes, which will allocate them for inclusion in the LVM scheme. This will write a header to the physical device and make them visible to the LVM process.
3. Use *vgcreate* to incorporate all of the desired physical volumes into a virtual collection called a volume group. The volume group now will act as a multi-disk equivalent of a physical volume on which partitioning can occur.
4. Use *lvcreate* to create the LVM version of disk partitions (called logical volumes) in the volume group created previously. The logical volumes act like partitions in that the user can create filesystems on them, mount them, and in general use them as a traditional partition.

The main advantages of using LVM in general and logical volumes, in particular, is the ability to group together space from multiple physical devices, resize them (as well as the filesystems on them), and much more.

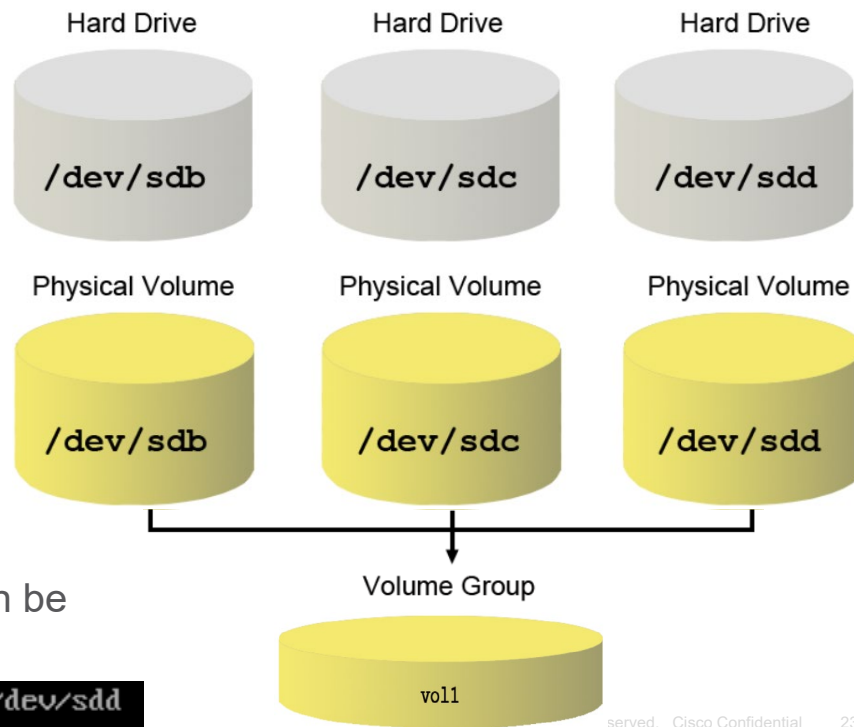
5.1 LVM Concepts (Cont'd)

- To develop a better understanding of how these steps work, consider a situation in which three new hard drives are added to the system: `/dev/sdb`, `/dev/sdc`, and `/dev/sdd`.
- At this point, they are just three hard drives that don't have anything on them, including a partition table. To use these as part of LVM, first execute the `pvcreate` command on each one:

```
[root@localhost ~]# pvcreate /dev/sdb
Physical volume "/dev/sdb" successfully created.
[root@localhost ~]# pvcreate /dev/sdc
Physical volume "/dev/sdc" successfully created.
[root@localhost ~]# pvcreate /dev/sdd
Physical volume "/dev/sdd" successfully created.
[root@localhost ~]#
```

- The next step is to create a volume group that consists of these three physical volumes. This can be accomplished with the following command:

```
[root@localhost ~]# vgcreate vol1 /dev/sdb /dev/sdc /dev/sdd
Volume group "vol1" successfully created
```



5.1 LVM Concepts (Cont'd)

- What this now means, is that all of the space from all three physical volumes can be used to create logical volumes. If /dev/sdb is 50GB, /dev/sdc is 20GB, and /dev/sdd is 10GB, a single logical volume could be created that is 80GB in size. Additionally, if another physical volume was added to the system (add a new hard drive, use the **pvcreate** command, and then a command called **vgextend**), then this new space could be used to create more logical volumes or add to the size of existing logical volumes.
- Any of the space in the vol1 volume group can be used to create a logical volume with a command like the following:

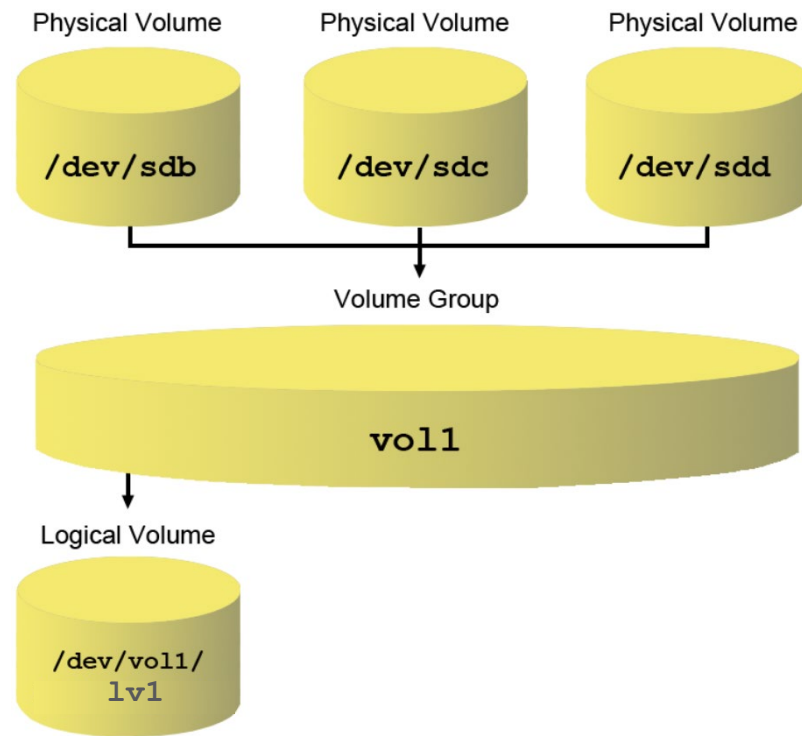
```
[root@localhost ~]# lvcreate -L 200M -n lv1 vol1  
Logical volume "lv1" created.
```

- The **-L** option is used to specify the size of the logical volume. The value of 200M means *create a 200MB logical volume*. The **-n** option is used to provide a name to the logical volume. The resulting name of the logical volume created by the previous command will be lv1. The last argument, vol1, is the name of the volume group from where the logical volume will obtain its physical space. The previous **lvcreate** command would result in a new device name of /dev/vol1/lv1 that could be used just like a traditional partition.

5.1 LVM Concepts (Cont'd)

- Resultant logical volume lv1 created as a result of the *lvcreate* command.
- To verify the outcome of the pvcreate, vgcreate and lvcreate commands, you can use pvdisplay, vgdisplay and lvdisplay.

```
[root@localhost ~]# lvdisplay
--- Logical volume ---
LV Path                /dev/vol1/lv1
LV Name                 lv1
VG Name                vol1
LV UUID                cmX5g4-PCmS-5Nm7-f3aA-Q673-L
LV Write Access         read/write
LV Creation host, time localhost.localdomain, 2019-
LV Status               available
# open                  0
LV Size                 200.00 MiB
Current LE              50
Segments                1
Allocation              inherit
Read ahead sectors      auto
- currently set to      8192
Block device            253:0
```



6 Creating a Filesystem

- The **mkfs** command is used to create a filesystem (or more commonly called formatting a partition/disk). To make a ext4 type filesystem, using the mkfs command as follows:

```
[root@localhost ~]# mkfs -t ext4 /dev/vol1/lv1
mke2fs 1.42.9 (28-Dec-2013)
Filesystem label=
OS type: Linux
Block size=1024 (log=0)
Fragment size=1024 (log=0)
Stride=0 blocks, Stripe width=0 blocks
51200 inodes, 204800 blocks
10240 blocks (5.00%) reserved for the super user
First data block=1
Maximum filesystem blocks=33816576
25 block groups
8192 blocks per group, 8192 fragments per group
2048 inodes per group
Superblock backups stored on blocks:
    8193, 24577, 40961, 57345, 73729

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done
```

7 Creating Swap Space

- Swap space (also called virtual memory) is hard drive space that can be used by the kernel and memory management routines to store data that is normally stored in RAM. When RAM starts to become full, the kernel will take some of this data and swap it to the hard drive. At a later time, as it's needed, the data will be swapped back to RAM.
- Even if the system has plenty of RAM, creating a swap space is still useful because if the system ever crashes, swap space is used to store a *crash dump file* which is used by advanced system administrators to determine why the system crashed. As a result, typically the size of the swap space is at least equal to the size of RAM.
- There are two types of swap spaces that can be created:
 - **Swap Partition:** The more common of the two, a swap partition is a partition that doesn't have a regular filesystem on it, and is not mounted. During installation, a swap partition is created, but additional swap partitions can be created at a later time.
 - **Swap File:** In the event that there is no unpartitioned space left on the hard drive, a swap file can be used. Swap partitions are typically faster than swap files as they are a filesystem, not a file, that is located on top of another filesystem. Swap files are more flexible and can be created on the fly without the need to repartition the hard drive.

7.1 Creating a Swap Partition

- The example on the right depicts the steps using fdisk to create a new partition of file system type Linux swap.
- The mkswap command is then run to convert the partition to swap space.

```
[root@localhost ~]# mkswap /dev/sdb1
mkswap: /dev/sdb1: warning: wiping old ext4 signature.
Setting up swspace version 1, size = 102396 KiB
no label, UUID=f3a25a81-b6d5-4c7c-84d4-2b038743532b
```

- This is followed by enabling the partition as current swap space with the swapon command.

```
[root@localhost ~]# swapon /dev/sdb1
[root@localhost ~]# swapon -s
```

Filename	Type	Size	Used	Priority
/dev/sda3	partition	2097148	0	-2
/dev/sdb1	partition	102396	0	-3

```
[root@localhost ~]# fdisk -c -u /dev/sdb
Welcome to fdisk (util-linux 2.23.2).

Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table
Building a new DOS disklabel with disk identifier 0xf3471c00.

Command (m for help): n
Partition type:
   p   primary (0 primary, 0 extended, 4 free)
   e   extended
Select (default p):
Using default response p
Partition number (1-4, default 1):
First sector (2048-2097151, default 2048):
Using default value 2048
Last sector, +sectors or +size(K,M,G) (2048-2097151, default 2097151): +100M
Partition 1 of type Linux and of size 100 MiB is set

Command (m for help): t
Selected partition 1
Hex code (type L to list all codes): 82
Changed type of partition 'Linux' to 'Linux swap / Solaris'
```

7.2 Creating a Swap File

- The example on the right depicts the steps to create a swap file of size 500MB. You need to make sure that there is sufficient space on the partition that you intend to create the swap file.

```
[root@localhost ~]# dd if=/dev/zero of=/extraswap bs=1M count=500
500+0 records in
500+0 records out
524288000 bytes (524 MB) copied, 3.78805 s, 138 MB/s
[root@localhost ~]# mkswap /extraswap
Setting up swspace version 1, size = 511996 KiB
no label, UUID=83d0af99-bdcf-496b-b807-47a1073891f4
```

```
[root@localhost ~]# swapon /extraswap
[root@localhost ~]# swapon -s
```

Filename	Type	Size	Used	Priority
/dev/sda3	partition	2097148	0	-2
/extraswap	file	511996	0	-3

Mounting Partitions

8 Introduction

- Linux treats almost everything as a file, which makes the job that the filesystem performs particularly important. The filesystem is a tree-like structure, with directories containing sub-directories and files. Unlike other operating systems you may be familiar with, the logical directory structure of Linux can span multiple disks and partitions, organized under a single root directory. Instead of manipulating drive letters, the administrator grafts, or *mounts*, new partitions, each with their own filesystem, on top of existing directories to give the illusion of a single, large, drive.
- Recall the three steps in the process of making and using filesystems:
 - Divide the hard drive into partitions.
 - Format each partition with a filesystem.
 - Mount the formatted partitions onto the directory tree of an existing filesystem.

9 View Mounted Filesystems

- You can use the mount command to view which are the partitions and filesystems currently mounted in the system. When executed with no arguments, the mount command shows the current devices mounted. The example list the output after piping the output of the mount command (which is quite lengthy) to the grep command.

```
[root@localhost ~]# mount | grep /dev/sda  
/dev/sda6 on / type ext4 (rw,relatime,seclabel,data=ordered)  
/dev/sda2 on /var type ext4 (rw,relatime,seclabel,data=ordered)  
/dev/sda1 on /boot type ext4 (rw,relatime,seclabel,data=ordered)  
/dev/sda5 on /home type ext4 (rw,relatime,seclabel,data=ordered)
```

The general format of this output is:

```
DESCRIPTION OF SOURCE on MOUNT POINT type FILESYSTEM TYPE (OPTIONS)
```

Description of Source

```
/dev/sda5 on /home type ext4 (rw)
```

```
proc on /proc type proc (rw)
```

9 View Mounted Filesystems (Cont'd)

- Every mounted filesystem comes from a source which can be a device (usb, cdrom, dvd or a sata disk partition etc), a network location for a network filesystem or a tag indicating a pseudo-filesystem (proc, sysfs, devpts, tmpfs etc). Such pseudo filesystem is Linux's way of sharing information with the real filesystem. Example, the proc filesystem exposes the process list and runtime system configuration as a set of files that can be read and written to.

Mount Point

```
/dev/sda2 on / type ext4 (rw)
/dev/sda5 on /home type ext4 (rw)
```

- The mount point shows where the source can be found on the filesystem. The output above indicates that the root /filesystem is actually served from the /dev/sda2 partition, but files in the /home directory are stored on the /dev/sda5 partition.

9 View Mounted Filesystems (Cont'd)

Filesystem Type

```
/dev/sda5 on /home type ext4 (rw)
proc on /proc type proc (rw)
```

- Each filesystem has a type, which tells the kernel how to work with the data contained on the device. In the example above, /dev/sda5 is an ext4 filesystem which tells the kernel that journaling needs to be supported and hence related modules need to be loaded when the device is mounted.
- File systems such as nfs refers to network filesystems that require network calls and packets sent and received to be translated into disk reads and writes.

9 View Mounted Filesystems (Cont'd)

Options

```
/dev/sda5 on /home type ext4 (rw)
devpts on /dev/pts type devpts (rw,gid=5,mode=620)
```

- Each filesystem also has options. Some options, such as the *read/write* `rw` option and *read-only* `ro` option, are fairly universal. Some filesystems have options corresponding to specific items such as the network timeout parameters for a network filesystem or the journalling parameters for a journaled filesystem.

9 View Mounted Filesystems (Cont'd)

- Another useful way to view the filesystem is by running the `lsblk` command. The command list information about block devices, either all of the ones that are available with the `-a` option (right), or output can be tailored with various other options such as `-f` option (below), which outputs information about available filesystems.

```
[root@localhost ~]# lsblk -f
```

NAME	FSTYPE	LABEL	UUID	MOUNTPOINT
sda				
├─sda1	ext4		728cdfc2-3ff8-45ac-87d6-6a5917c314a3	/boot
├─sda2	ext4		962b38c7-521e-47d5-a40f-b6bf604aaec6	/var
├─sda3	swap		2f40f568-4bc6-45e4-af05-ab28c56474a7	[SWAP]
├─sda4				
├─sda5	ext4		d80d98c0-1533-47f7-8e62-17e5643b9615	/home
└─sda6	ext4		774392c7-624d-4034-acae-9c5314e00019	/
sdb				
sdc				
sdd				
sr0				

```
[root@localhost ~]# lsblk -a
```

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINT
sda	8:0	0	8G	0	disk	
├─sda1	8:1	0	500M	0	part	/boot
├─sda2	8:2	0	2G	0	part	/var
├─sda3	8:3	0	2G	0	part	[SWAP]
├─sda4	8:4	0	1K	0	part	
├─sda5	8:5	0	1G	0	part	/home
└─sda6	8:6	0	2.5G	0	part	/
sdb	8:16	0	1G	0	disk	
sdc	8:32	0	1G	0	disk	
sdd	8:48	0	1G	0	disk	
sr0	11:0	1	1024M	0	rom	

10 Mounting Filesystems Manually

- In order to properly mount a filesystem using the mount command, the filesystem identifier and the mount point (directory path name) must be provided. Other parameters may be provided to specify the mount options to use and the filesystem type.
- Mounting the filesystem manually requires root privileges.

```
[student@localhost ~]$ mount /dev/sdb1 /mnt
mount: only root can do that
[student@localhost ~]$ su -
Password:
Last login: Tue Dec  3 20:02:19 EST 2019 on tty1
[root@localhost ~]# mount /dev/sdb1 /mnt
[root@localhost ~]# mount | grep /dev/sdb1
/dev/sdb1 on /mnt type ext4 (rw,relatime,seclabel,data=ordered)
```

- Most often then not, the mount point must be created first before you can mount a device to it. Else, the /mnt and /media (/cdrom in CentOS) may be used as they exist by default on most Linux distributions.
- In the above example, Linux automatically detects the filesystem as ext4 and mounts it using default mount options. If the filesystem cannot be detected, you can use the -t option to indicate it.

10 Mounting Filesystems Manually (Cont'd)

- In the above example, we use the -t option to mount a cdrom.
- If you wish to mount a particular device using options other than the defaults, use the -o option. In the example below, /dev/sdb1 has been mounted as read-only. Note the failure to touch the file test.txt.
- It is a good practice to unmount a device before removing the device from the system (eg, usb). Notice that the unmounting will be unsuccessful if your current working directory is within the mount point.

```
[root@localhost ~]# umount /mnt
[root@localhost ~]# mount -t iso9660 /dev/cdrom /mnt
mount: /dev/sr0 is write-protected, mounting read-only
[root@localhost ~]#
[root@localhost ~]# ls /mnt
readme.txt
```

```
[root@localhost ~]# mount /dev/sdb1 -o ro /mnt
[root@localhost ~]# cd /mnt
[root@localhost mnt]# touch test.txt
touch: cannot touch 'test.txt': Read-only file system
```

```
[root@localhost mnt]# umount /mnt
umount: /mnt: target is busy.
(In some cases useful info about processes that use
the device is found by lsof(8) or fuser(1))
[root@localhost mnt]# cd
[root@localhost ~]# umount /mnt
[root@localhost ~]# mount | grep /mnt
```

11 Unmounting Filesystems Manually

- You can unmount a filesystem either by specifying the mount point (eg. /mnt) or the device (eg, /dev/sdb1) as an argument to the umount command.
- In the unlikely event that some files on the device are currently used by system processes, you can use the lsof command to list the open files on the operating system.

```
[root@localhost ~]# umount /home
umount: /home: target is busy.
(In some cases useful info about processes that use
the device is found by lsof(8) or fuser(1))
```

```
[root@localhost ~]# lsof | grep /home
bash      3812      student  cwd          DIR           8,5      4096      16 /home/stu
dent
bash     31040      student  cwd          DIR           8,5      4096      16 /home/stu
dent
```

- After terminating the processes that are holding on to the open files, the umount is now successfully completed.

```
[root@localhost ~]# lsof | grep /home
[root@localhost ~]# umount /home
```

```
sysdamin@linux:~$ sudo fuser -u -v /
student    3021 .r... (student)ping
```

- You may also use the fuser command on Ubuntu systems as shown above on the right (partial output).

12 Mounting Filesystems Automatically On Boot

- Mounting filesystems (including activating swap) manually results in non-persistent mounts. If the system is rebooted, then the filesystem must be mounted again.
- The `/etc/fstab` file is used to configure what filesystems will be mounted automatically at boot. It can also be used to activate swap devices automatically. It may be helpful to think of this file as the *filesystem table*, to remember its name. To those familiar with mounting a filesystem manually, the format of the file should make sense.
- Root privileges are required to make changes to the `/etc/fstab` file. Any changes made to this file should be performed with care, as mistakes may prevent the system from booting normally.

```
##
## /etc/fstab
## Created by anaconda on Mon Dec  2 01:50:08 2019
##
## Accessible filesystems, by reference, are maintained under '/dev/disk'
## See man pages fstab(5), findfs(8), mount(8) and/or blkid(8) for more info
##
UUID=774392c7-624d-4034-acae-9c5314e00019 /          ext4    defaults    1 1
UUID=728cdf2e-3ff8-45ac-87d6-6a5917c314a3 /boot      ext4    defaults    1 2
UUID=d80d98c0-1533-47f7-8e62-17e5643b9615 /home      ext4    defaults    1 2
UUID=962b38c7-521e-47d5-a40f-b6bf604aaec6 /var       ext4    defaults    1 2
UUID=2f40f568-4bc6-45e4-af05-ab28c56474a7 swap       swap    defaults    0 0
```

12 Mounting Filesystems Automatically On Boot (Cont'd)

- The first column in the `/etc/fstab` file identifies the device to be mounted. This could be in the form of `/dev/sda1` but more commonly, the UUID is listed instead. The best practice is to use UUID instead of device name as the latter is unpredictable as the system is unattended during boot time and the name may be swapped simply through a change of physical SATA cables. Identifying the device using the UUID does not have this issue.

UUID

For this reason, the most common way to refer to a partition is by using a *universally unique identifier (UUID)*, as in the following example:

```
UUID=3db6ba40-67d2-403d-9c0a-9a901697cd8d /      ext4    defaults    1 1
UUID=09d641d5-bc5a-4065-8d80-8ae797dfa7f3 /boot  ext4    defaults    1 2
UUID=5ee634a5-c360-4211-a41b-9aa40d78a804 /home  ext4    defaults    1 2
```

- You can find out the UUID for a device by using the `blkid` command

```
[root@localhost ~]# blkid
/dev/sdb1: UUID="95de528f-28c8-40ee-a382-bb1a9005e381" TYPE="ext4"
/dev/sda1: UUID="728cdfc2-3ff8-45ac-87d6-6a5917c314a3" TYPE="ext4"
/dev/sda2: UUID="962b38c7-521e-47d5-a40f-b6bf604aaec6" TYPE="ext4"
/dev/sda3: UUID="2f40f568-4bc6-45e4-af05-ab28c56474a7" TYPE="swap"
/dev/sda5: UUID="d80d98c0-1533-47f7-8e62-17e5643b9615" TYPE="ext4"
/dev/sda6: UUID="774392c7-624d-4034-acae-9c5314e00019" TYPE="ext4"
```

12 Mounting Filesystems Automatically On Boot (Cont'd)

- The UUID is automatically created when the filesystem is created. It is considered the best way to identify local filesystems in the `/etc/fstab` file because it doesn't change after the filesystem has been created and it should always be unique. Even if the hard drive is moved to another computer, the UUIDs will be unique for each filesystem. Unfortunately, the UUID isn't easy to remember or type, so it is difficult to use for manual mounting.

Device Name

A second, more traditional way of specifying a local filesystem is to use the path name for the device, such as `/dev/sda1` for the first partition on the first disk:

<code>/dev/sda1</code>	<code>/</code>	<code>ext4</code>	<code>defaults</code>	<code>1 1</code>
<code>/dev/sda2</code>	<code>/boot</code>	<code>ext4</code>	<code>defaults</code>	<code>1 2</code>
<code>/dev/sda5</code>	<code>/home</code>	<code>ext4</code>	<code>defaults</code>	<code>1 2</code>

- While device path names like `/dev/sdb1` are easy to remember and type, they are not as reliable as UUIDs. As the naming of the disk drive depends on the order of detection during the boot process, the naming of the disks might change if the order of detection changes (such as when a new disk drive is added to the system).

12 Mounting Filesystems Automatically On Boot (Cont'd)

- The following table list the two methods to specify the device identifier in the /etc/fstab file:

Method	Advantages	Disadvantages
UUID	Should always be unique, preventing any mounting conflicts.	Hard to type and difficult to remember.
Device Name	Easy to type and remember.	Device names can easily change, causing mounting errors.

- Note that there is one more method which is to use labels which is not required in this course.

12 Mounting Filesystems Automatically On Boot (Cont'd)

- The next field on the right is the mount point where the partition is to be mounted.

```
UUID=3db6ba40-67d2-403d-9c0a-9a901697cd8d / ext4 defaults 1 1
UUID=09d641d5-bc5a-4065-8d80-8ae797dfa7f3 /boot ext4 defaults 1 2
UUID=5ee634a5-c360-4211-a41b-9aa40d78a804 /home ext4 defaults 1 2
```

- The third field is the filesystem field. It can be anyone of the following:

Name	Type Code
Fourth Extended Filesystem	ext4
Third Extended Filesystem	ext3
Second Extended Filesystem	ext2
Extents Filesystem	xfs
File Allocation Table	vfat
ISO 9660	iso
Universal Disc Format	udf

Refer to the example above, most of the time the filesystem will be ext4.

12 Mounting Filesystems Automatically On Boot (Cont'd)

- The fourth field is the mount option field. The table below lists the common options.

```
UUID=3db6ba40-67d2-403d-9c0a-9a901697cd8d /date      ext4      ro      1 1
```

Mount Option	Purpose	Opposite
rw	Allow reading and writing	ro
suid	Allow suid executes	nosuid
dev	Allow device files	nodev
exec	Allow executable files	noexec
auto	Automatically mount	noauto
nouser	Prevent ordinary users from mounting or unmounting the partition. Using the user option allows non-root users to mount that particular device, which is helpful for removable media.	user
async	All writes should be asynchronous	sync
relatime	Only update access time on file access if the file has been modified or its metadata changed since last access	N/A

12 Mounting Filesystems Automatically On Boot (Cont'd)

- The next two fields are the dump field and the filesystem check field which is normally set to 0 and 0 respectively.
- Except for the root filesystem, all other devices should be set to 0 and 0 for simplicity. Deep knowledge of these last two fields is not required for this course.

13 Mount a Filesystem

- The example on the right summarises the steps to automatically mount a newly formatted SATA hard drive using the /etc/fstab. The /dev/sdb has only one ext4 partition /dev/sdb1 and will be mounted to the mount point /mnt/sdb1 everytime the system reboots.

```
[root@localhost ~]# tail -n 5 /etc/fstab
UUID=774392c7-624d-4034-acae-9c5314e00019 / ext4 defaults 1 1
UUID=728cdf2-3ff8-45ac-87d6-6a5917c314a3 /boot ext4 defaults 1 2
UUID=d80d98c0-1533-47f7-8e62-17e5643b9615 /home ext4 defaults 1 2
UUID=962b38c7-521e-47d5-a40f-b6bf604aaec6 /var ext4 defaults 1 2
UUID=2f40f568-4bc6-45e4-af05-ab28c56474a7 swap swap defaults 0 0
[root@localhost ~]# echo "$(blkid /dev/sdb1 | cut -d' ' -f2) /mnt/sdb1 ext4 defaults 0 0" >> /etc/fstab
[root@localhost ~]# tail -n 6 /etc/fstab
UUID=774392c7-624d-4034-acae-9c5314e00019 / ext4 defaults 1 1
UUID=728cdf2-3ff8-45ac-87d6-6a5917c314a3 /boot ext4 defaults 1 2
UUID=d80d98c0-1533-47f7-8e62-17e5643b9615 /home ext4 defaults 1 2
UUID=962b38c7-521e-47d5-a40f-b6bf604aaec6 /var ext4 defaults 1 2
UUID=2f40f568-4bc6-45e4-af05-ab28c56474a7 swap swap defaults 0 0
UUID="c58976ff-42f9-4e4c-b734-8e247ebfc35c" /mnt/sdb1 ext4 defaults 0 0
[root@localhost ~]# mount -a
[root@localhost ~]# lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINT
sda 8:0 0 8G 0 disk
├─sda1 8:1 0 500M 0 part /boot
├─sda2 8:2 0 2G 0 part /var
├─sda3 8:3 0 2G 0 part [SWAP]
├─sda4 8:4 0 512B 0 part
├─sda5 8:5 0 1G 0 part /home
└─sda6 8:6 0 2.5G 0 part /
sdb 8:16 0 8G 0 disk
└─sdb1 8:17 0 8G 0 part /mnt/sdb1
sr0 11:0 1 1024M 0 rom
```

14 Activating Swap Space

- The following example illustrate the entry in /etc/fstab to activate the swap space persistently when the /dev/sdb2 is configured to be the swap device.

```
[root@localhost ~]# blkid /dev/sdb2
/dev/sdb2: UUID="7558ef9c-ed83-4322-a956-85159218dbd5" TYPE="swap"
```

```
#
# /etc/fstab
# Created by anaconda on Mon Dec  2 01:50:08 2019
#
# Accessible filesystems, by reference, are maintained under '/dev/disk'
# See man pages fstab(5), findfs(8), mount(8) and/or blkid(8) for more info
#
UUID=774392c7-624d-4034-acae-9c5314e00019 /                ext4      defaults    1 1
UUID=728cdfc2-3ff8-45ac-87d6-6a5917c314a3 /boot            ext4      defaults    1 2
UUID=d80d98c0-1533-47f7-8e62-17e5643b9615 /home            ext4      defaults    1 2
UUID=962b38c7-521e-47d5-a40f-b6bf604aaec6 /var              ext4      defaults    1 2
UUID=2f40f568-4bc6-45e4-af05-ab28c56474a7 swap              swap      defaults    0 0
UUID=7558ef9c-ed83-4322-a956-85159218dbd5 swap              swap      defaults    0 0
```

```
-----
/dev/sda3      partition    2097148 0      -2
/dev/sdb2      partition    1048572 0      -3
```

14 Activating Swap Space (Cont'd)

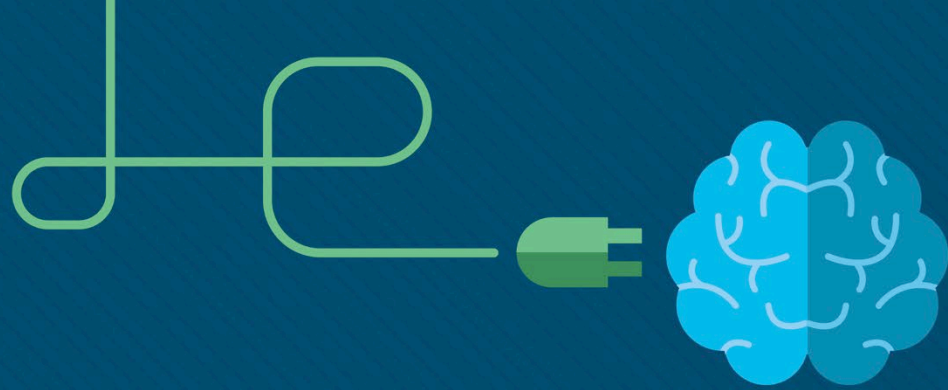
- After rebooting the machine, you can check that the swap space /dev/sdb2 is automatically activated

```
[root@localhost ~]# swapon -s
```

Filename	Type	Size	Used	Priority
/dev/sda3	partition	2097148	0	-2
/dev/sdb2	partition	1048572	0	-3

- The second example below shows how to automatically mount a swap file instead of a swap partition device

```
#  
# /etc/fstab  
# Created by anaconda on Mon Dec  2 01:50:08 2019  
#  
# Accessible filesystems, by reference, are maintained under '/dev/disk'  
# See man pages fstab(5), findfs(8), mount(8) and/or blkid(8) for more info  
#  
UUID=774392c7-624d-4034-acae-9c5314e00019 / ext4 defaults 1 1  
UUID=728cdfc2-3ff8-45ac-87d6-6a5917c314a3 /boot ext4 defaults 1 2  
UUID=d80d98c0-1533-47f7-8e62-17e5643b9615 /home ext4 defaults 1 2  
UUID=962b38c7-521e-47d5-a40f-b6bf604aaec6 /var ext4 defaults 1 2  
UUID=2f40f568-4bc6-45e4-af05-ab28c56474a7 swap swap defaults 0 0  
/var/swapfile swap swap defaults 0 0
```



Lab

RH134 Chapter 7.5 Lab Manage Basic Storage
RH134 Chapter 8.5 Lab Manage Storage Stack



Practice Exercise (40 minutes) - Partitioning

2 GiB SATA Disk

2 GiB SATA Disk					
400 MiB	450 MiB	100 MiB	Balance of Disk Storage - Extended		
Primary	Primary	Primary	200 MiB	300 MiB	Balance of Extended Storage
Linux	Linux	Linux Swap	Logical	Logical	Logical
ext4	ext4	swap	LVM	LVM	LVM
/mnt/sdb1	/mnt/sdb2				

Practice Exercise (30 minutes) - LVM

